

Critical Audit of SZ-DBC (Second Report): Detailed Balance Checker for Lattice MCMC

Correctness, rigor, and portability analysis
Incorporating all fixes from the first report

Independent Technical Review

June 2026

Contents

1	Scope and Method	3
2	The BFS Engine: Correctness Review	3
2.1	Weight completeness	3
2.2	RandomChoice correction for non-power-of-2 pool sizes	4
2.3	Edge cases in the BFS engine	4
3	The EBE Exact DB Check	4
3.1	Phase 1: Condition extraction	4
3.2	Phase 2: Chamber enumeration	5
3.3	Phase 3: The <i>\$dbcIsExpZero</i> check	5
4	The D4 Algebraic Check: Review of the New Implementation	6
4.1	Generator sufficiency	6
4.2	Correctness of Phase 3: checking $T(i \rightarrow j)$ vs $T(R \cdot i \rightarrow R \cdot j)$	6
4.3	Chamber reuse: a correctness assumption	6
4.4	C4-only case	7
5	The Translational Invariance Check	8
5.1	Correctness of weight-only check	8
5.2	The <i>cantHandle</i> path for <i>quadratic_field.wl</i>	8
5.3	Soundness of checking from orbit representatives	8
6	Ergodicity Check	8
7	New and Undocumented Issues	9
7.1	NormalDistribution discretisation: undocumented truncation error	9
7.2	Coupling atom arity constraint	9
7.3	Legacy dead code	9
7.4	RandomInteger with inverted bounds	10

8	Correctness of Orbit Expansion	10
8.1	The orbit map construction	10
8.2	Orbit expansion correctness	10
8.3	Self-loops and T-matrix diagonal	10
9	Summary of Open Issues	10
10	Performance Analysis: The D4 Orbit-Reduction Trade-off	11
10.1	Background and motivation	11
10.2	Timing breakdown with and without D4	11
10.3	Why D4 orbit reduction barely helps Phase 3	12
10.4	Why the D4 check appeared cheap (a subtle asymmetry)	12
10.5	Implication: D4 is not a performance optimisation for dense transition graphs	13
10.6	The probabilistic fallback: permanently disabled	13
11	Python and Open-Source Portability Analysis	13
11.1	Motivation and scope	13
11.2	The five core mechanisms and their portability	13
11.2.1	Mechanism 1: Dynamic function interception (Block)	13
11.2.2	Mechanism 2: Operator overloading for comparison tokens	14
11.2.3	Mechanism 3: Symbolic algebra for Piecewise expressions	14
11.2.4	Mechanism 4: State enumeration and orbit computation	15
11.2.5	Mechanism 5: EBE Phase 2 chamber enumeration	15
11.3	Component-by-component portability summary	15
11.4	What the removal of FullSimplify changes	15
11.5	A realistic Python port: what would it look like?	16
11.6	Julia: the recommended open-source alternative	17
11.7	C++: not recommended	18
11.8	The fundamental limitation: algorithm transparency	18
12	Recommendations	18
12.1	Immediate fixes	18
12.2	Documentation	19
12.3	For a Python or Julia port	19
13	Conclusions	19

1 Scope and Method

This report constitutes a deep, independent audit of the SZ-DBC codebase as it stands after the two rounds of fixes described in the previous report (`critical_report.tex`). All seven issues raised there have been addressed; this review therefore starts fresh and examines the *current* code for new correctness problems, rigour gaps, and undocumented limitations.

The methodology was: read every line of `dbc_core.wl` and `check.wls`; run all provided examples and verify their outputs; run targeted Wolfram Language micro-experiments to probe specific claims; and reason from first principles about every non-trivial assertion made by the code or its documentation. All example runs were performed on the current codebase with `wolframscript`.

Top-line verdict. The checker is now substantially more rigorous than the version audited in the first report. The critical RandomChoice weight bug is fixed, the D4 check is algebraically exact, the ergodicity BFS starts from the correct seed, and timed-out BFS paths abort with an error. The EBE-based DB check is exact for all algorithms with fewer than `ebeMaxK` branch conditions.

Two moderate correctness concerns survive: (1) an unverified assumption that the post-D4 BFS introduces no new Piecewise conditions beyond those seen in the pre-D4 BFS, silently relied upon when reusing EBE chambers; (2) the `$dbcFS` sentinel is treated as a DB violation rather than an analysis failure. Neither causes problems for the provided examples. Three minor issues round out the analysis.

2 The BFS Engine: Correctness Review

2.1 Weight completeness

A necessary condition for BFS correctness is that the leaf weights for any single starting state sum to 1 (probability is conserved across all execution paths). This was verified experimentally on the first orbit representative of `single_metropolis.wl`:

```
Total[#[[3]] & /@ leaves] (* symbolic sum over all leaves *)
(* After Simplify: 1 *)
```

The sum is a large symbolic expression involving Piecewise coupling comparisons and Boltzmann factors; `Simplify` reduces it to 1 exactly. This confirms that the accept/reject branches pair correctly and that the weight-correction factors $2^k/n$ for `RandomChoice` are applied consistently. The proof that weight sums to 1 holds generally:

Theorem 1 (Weight completeness). For any algorithm whose random calls are all handled by the BFS engine (no `cantHandle`), the leaf weights from any starting state sum to 1.

Sketch. By construction, each `RandomReal[]` comparison branches into two paths with weights p and $1 - p$ summing to 1. Each `RandomChoice[list]` with n elements reads $k = \lceil \log_2 n \rceil$ bits; in-range leaves each have weight $2^k/n$ (after correction), and the $2^k - n$ out-of-range paths are discarded. The n in-range leaves carry weight $n \cdot (1/2^k) \cdot (2^k/n) = 1$. The `seqBernoulli` decomposition is a perfect partition of $[0, 1]$ by construction (verified above). The product of independent branches gives a probability measure on the leaf set. $\square$

2.2 RandomChoice correction for non-power-of-2 pool sizes

This was the critical bug fixed in the first revision. The current code applies:

```
weight *= 2^k/n; (* after selecting valid index *)
```

for `RandomChoice[list]`, `RandomInteger[{lo,hi}]`, and the vectorised forms. For n a power of 2 the factor is 1 and has no effect; for general n it corrects the per-leaf weight from $1/2^k$ to $1/n$. The broken examples `broken_variable_pool.wl` (pool size 3 or 4, $k = 2$) and `broken_8way_hop.wl` (pool size 7 or 8, $k = 3$) now correctly report **FAIL** with exact rational transition probabilities printed.

Verified 1. All nine examples produce the expected verdict. D4 is no longer declared in `$symmetryGroup` by default (see Section 10): the three passing examples (`single_metropolis`, `kawasaki`, `vmmc_2d`) show D4 rotational: not declared in their summary output. DB correctness is unaffected.

2.3 Edge cases in the BFS engine

Empty range `RandomInteger[{hi,lo}]` with $hi < lo$. When $n = hi - lo + 1 \leq 0$, the code computes $k = \text{IntegerLength}[n - 1, 2]$. For $n = 0$: `IntegerLength[-1, 2] = 1`, so one bit is read and $val \in \{0, 1\}$; both satisfy $val \geq n = 0$, so `$dbc$outOfRange` is thrown and the path is silently discarded. This is correct behaviour (a call with inverted bounds has no valid outcomes) but is not documented.

Single-element lists. `RandomChoice[{x}]` ($n=1$) returns x immediately without reading bits (explicit `n==1` branch). `RandomInteger[{lo,lo}]` ($n=1$) similarly returns `lo` with weight 1. `seqBernoulli[... , {e}]` (one element) is short-circuited before the call site. All correct.

Zero-weight seqBernoulli guard. The guard `If[TrueQ[remainW == 0], Throw[$dbc$cantHandle]` was added in the second revision. Experimentally verified: for concrete zeros (`ws = {0,0,0}`) and for symbolically simplified zeros (`{couplingJ[1,2,1], -couplingJ[1,2,1]}`), Mathematica simplifies `Total[ws]` to 0 and `TrueQ[0 == 0]` returns `True`, so the guard fires. For genuinely pathological cases where a symbolic total equals zero but Mathematica does not simplify it, division by zero would produce `Indeterminate`. This is unlikely for physical algorithms (weights are non-negative energy-difference expressions).

3 The EBE Exact DB Check

3.1 Phase 1: Condition extraction

The extraction pattern matches `HoldPattern[Piecewise[cl_List, _]]` to avoid premature evaluation of `Piecewise` clauses. It retains only `_Less` | `_LessEqual` | `_Greater` | `_GreaterEqual` conditions that involve at least one symbolic parameter. This correctly excludes `True`, `False`, `=== Infinity` sentinels (`SameQ`, not a comparison), and numeric constants.

For `vmmc_2d.wl`, link probabilities include clauses of the form `{1, eFwd === Infinity}` using `===` (structural equality), which are not matched by the comparison filter and are therefore never extracted as EBE conditions. Since `couplingJ[... ] === Infinity` evaluates to `False` for any finite coupling, these clauses always contribute weight 0 and the extraction is correct.

3.2 Phase 2: Chamber enumeration

Claim 1 (Correctness of arrangement BFS). The chamber-adjacency BFS in Phase 2 visits every non-empty open chamber of the hyperplane arrangement defined by the k Piecewise conditions.

Proof. The arrangement is finite; each hyperplane is defined by a linear inequality in the coupling constants. By the standard connectivity theorem (any two chambers are path-connected via a sequence of codimension-1 face crossings), the face-adjacency graph is connected. The BFS therefore visits every chamber reachable from the starting point. By connectivity, this is all chambers.

For each adjacent chamber (sign-pattern obtained by flipping one condition), `FindInstance[constraints, allSymParams, Rationals, 1]` is called. For rational-coefficient linear inequalities over the reals, a rational interior point exists whenever the region is non-empty (rational points are dense in any open set). Thus if a chamber is non-empty, `FindInstance` will find a representative. $\square$

Boundary omission. The open-chamber enumeration does not test the lower-dimensional faces where two or more conditions are simultaneously tight (e.g., $J_1 = J_2$ exactly). A DB violation existing only on such a boundary (and not in any adjacent open chamber) would be missed. For Metropolis-type algorithms, transition probabilities are continuous functions of the coupling constants; any violation in a positive-measure region must therefore appear in an adjacent open chamber. The only pathological case is an algorithm with an explicit equality-trigger rule (e.g. `If [ $J_1 == J_2$ , ...]`), which would be non-standard. This should be documented.

3.3 Phase 3: The `$dbcIsExpZero` check

After substituting a rational representative J^* into every leaf weight, the DB expression takes the form $\sum_i c_i e^{-\beta E_i}$ where c_i are rational and E_i are rational multiples of β , i.e. $\sum_i c_i E^{-\beta r_i}$ with $r_i \in \mathbb{Q}$. The function `$dbcIsExpZero` checks this is zero as follows:

1. `$dbcMergeExp`: iteratively replaces $E^a \cdot E^b \rightarrow E^{a+b}$ until no such pairs remain (at most 30 iterations).
2. `$dbcSplitTerm`: for each term, extract the unique Exp factor (if any) to produce `{coefficient, exponent}`. Returns `$dbcFS` if two or more distinct Exp factors remain after merging.
3. `$dbcGroupByExp`: group terms by exponent using `Expand[...]` `=== 0`.
4. Check that the sum of rational coefficients in each group is zero.

Correctness for standard algorithms. Experimentally verified: after J^* substitution, all Exp factors in Metropolis-type algorithms are of the form $E^{-\beta r}$; `$dbcMergeExp` reduces any product $E^{-\beta r_1} \cdot E^{-\beta r_2}$ in one iteration. The `$dbcFS` branch is never triggered for any of the provided examples. For algorithms producing genuine transcendental coefficients (e.g. $E^{\sin(\beta)}$), merging would fail after 30 iterations and `$dbcFS` would be returned.

Issue 1 (`$dbcFS` treated as a violation, not an analysis failure). The check `If[res != True, violations[pair] = ...]` treats both *False* (genuine DB violation) and `$dbcFS`

(analysis failure) as violations. A user whose algorithm produces a `$dbcFS` result (e.g. an algorithm using $\sin(\beta)$ in the acceptance probability) would receive a `FAIL` verdict with no diagnostic distinguishing the two cases. The violation record would contain a `tij` or `tji` field with an unresolved symbolic expression, which is the only hint that analysis failed rather than `DB` being violated.

This is a pre-existing issue from the first report, not yet fixed. It is harmless for all standard physics algorithms tested, but a defensive fix (checking `res == $dbcFS` separately and aborting with an error) is straightforward.

4 The D4 Algebraic Check: Review of the New Implementation

4.1 Generator sufficiency

The D4 check tests only two generators: `rotIdx=1` (90° clockwise rotation) and `rotIdx=4` (left-right reflection). The claim is that these generate the full D4 group. This is verified:

Proof. With $r = \text{rot90CW}$ (order 4) and $s = \text{reflLR}$ (order 2), we have:

$$\begin{aligned} r^0 &= \text{id}, & r^1 &= \text{rot90}, & r^2 &= \text{rot180}, & r^3 &= \text{rot270} \\ s \cdot r^0 &= \text{reflLR}, & s \cdot r^1 &= \text{reflDiag}, & s \cdot r^2 &= \text{reflUD}, & s \cdot r^3 &= \text{reflAnti} \end{aligned}$$

These 8 elements are all distinct and cover D4. Verified numerically by applying all compositions to an asymmetric test state and confirming 8 distinct images matching all `rotIdx` 0–7. $\square$

The group-theoretic argument that verifying invariance for generators r and s implies invariance for all products is correct by induction: if $T(x \rightarrow y) = T(r \cdot x \rightarrow r \cdot y)$ for all pairs, then applying this to the pair $(s \cdot x, s \cdot y)$ gives $T(s \cdot x \rightarrow s \cdot y) = T(r \cdot s \cdot x \rightarrow r \cdot s \cdot y)$, establishing invariance under $r \cdot s$. By induction over all words in $\{r, s\}$, invariance holds for all D4 elements.

4.2 Correctness of Phase 3: checking $T(i \rightarrow j)$ vs $T(R \cdot i \rightarrow R \cdot j)$

In Phase 3 of `$dbcEBECheckD4`, the transition $T(R \cdot i \rightarrow R \cdot j)$ is looked up via `pairToLeafSrc[{rsi, rsj}]`, where `rsi` and `rsj` are the indices of $R \cdot i$ and $R \cdot j$ in `allStates`. The orbit expansion from the translation-only BFS covers all pairs in `allStates` (since the translation group generates all n^2 positions). Therefore $T(R \cdot i \rightarrow R \cdot j)$ is correctly assembled from the expanded leaves. If $T(R \cdot i \rightarrow R \cdot j) = 0$ and $T(i \rightarrow j) \neq 0$, the check `$dbcIsExpZero[tij - 0]` correctly detects the violation.

The check in Phase 3 evaluates the difference $T(i \rightarrow j) - T(R \cdot i \rightarrow R \cdot j)$ using `$dbcIsExpZero`. Since both T values are sums of rational $\times E^{-\beta r}$ terms (after J^* substitution), the difference has the same structure. The checker handles this correctly.

4.3 Chamber reuse: a correctness assumption

Issue 2 (Chamber reuse assumption not verified). After D_4 passes, the pipeline recomputes orbit representatives using the full D_4 +translation group (reducing from 56 to 8 reps for `single_metropolis`) and re-runs the BFS from these 8 reps. The EBE DB check then

reuses the 132 chambers computed during the D_4 EBE check (which used the 56-rep BFS), skipping Phase 2.

The correctness of this reuse relies on the following assumption:

Reuse assumption. Every Piecewise condition appearing in the 8-rep BFS leaves is also a condition in the 56-rep BFS leaves. Equivalently, the 8-rep hyperplane arrangement is a coarsening of (or identical to) the 56-rep arrangement.

If this assumption fails, some 8-rep Piecewise conditions are not controlled by the chamber decomposition; the J^* for each chamber fixes them to specific values, but those values may not be representative of the entire chamber. The DB check would be valid only at the single point J^* , not throughout the chamber.

Mathematical justification. If D_4 invariance holds exactly (as verified by the algebraic D_4 EBE check), then any energy-difference comparison arising in the 8-rep BFS also arises in the 56-rep BFS (or is D_4 -equivalent to one that does, and since D_4 acts on positions and not types, the coupling atoms are preserved). The 8-rep conditions are therefore a subset of the 56-rep conditions, making the reuse valid.

Empirical verification. In all provided examples, the EBE Phase 1 applied to the 8-rep BFS yields the same k as Phase 1 of the D_4 EBE check:

- `single_metropolis.wl`: $k = 18$ for both checks, 132 chambers
- `ummc_2d.wl`: $k = 12$ for both checks, 512 chambers
- `kawasaki.wl`: $k = 6$ for both checks, 12 chambers

What is missing. The code does not assert that the 8-rep conditions form a subset of the 56-rep conditions. A defensive check would be:

```
(* After Phase 1 of DB EBE, if reusing precomputedChambers: *)
If [!SubsetQ[allConds, precomputedConditions],
  Print["WARNING: new conditions not covered by precomputed
        chambers"];
  (* fall back to Phase 2 *)]
```

Without this check, a future algorithm where D_4 invariance is algebraically verified but the 8-rep BFS introduces new Piecewise conditions (e.g., from a distance function the 56-rep never reached) would silently produce an incomplete DB check. The mathematical argument above rules this out for standard physics algorithms, but a runtime assertion is prudent for a tool claiming exactness.

4.4 C4-only case

The same reuse concern applies when only C4 (rotations, not reflections) is verified. In this case, the post-C4 BFS uses C4+translation orbits (between 8 and 56 reps, depending on the system). The D_4 EBE check computed chambers from the 56-rep BFS. For C4-symmetric algorithms without full D_4 , it is possible (though unlikely for standard Hamiltonians) that the post-C4 BFS introduces conditions absent from the 56-rep BFS.

5 The Translational Invariance Check

5.1 Correctness of weight-only check

The τ -BFS verifies that leaf weights are free of τ_r and τ_c . It does not verify that the next-state for each leaf is the correct τ -shifted version of the non-augmented next state. For algorithms where spatial decisions use only pairwise differences, the state update (e.g., moving a particle from p to $p + \Delta$) always produces states that correctly carry τ . A pathological algorithm could, in principle, produce τ -free weights while constructing next states that are not translates of the rep's next states. Such an algorithm would pass the τ -check while being non-translation-invariant. No such algorithm is physically meaningful, but the limitation should be noted.

5.2 The cantHandle path for quadratic_field.wl

Confirmed from the current run: `quadratic_field.wl` triggers an error

```
RandomInteger[{{-16,16},4}]: 24 bits needed --- use count x
RandomInteger[{lo,hi}]
```

and falls back to the non-translation-invariant path. This is the improved error message from the second revision. The actual mechanism is that Mathematica's symbolic evaluation of `Mod[pos, n]` with a τ -augmented position internally calls `RandomInteger[{-16,16}, 4]`, which is intercepted. The checker correctly falls back and the DB check is exact.

5.3 Soundness of checking from orbit representatives

Theorem 2 (Sufficiency of orbit representatives for τ -check). If the τ -BFS produces τ -free leaf weights for each translation-orbit representative, it produces τ -free weights for every state.

Proof. Every state s is a translation $T_{(dr,dc)}(\text{rep})$ for some $(dr,dc) \in \{0, \dots, n-1\}^2$. Running the algorithm from $s + \tau = T_{(dr,dc)}(\text{rep}) + \tau = T_{(dr,dc)}(\text{rep} + \tau)$ (where $\tau = (\tau_r, \tau_c)$) is equivalent to running from $\text{rep} + \tau'$ with $\tau' = \tau + (dr, dc)$. Since the τ -BFS checks rep at offset τ (a free variable), the output subsumes all integer offsets of τ . Hence τ -freedom for the rep implies τ -freedom for all states in its orbit. $\square$

6 Ergodicity Check

The ergodicity BFS was fixed in the first revision to start from the correct `seedState` index. The current implementation looks up `seedIdx = Lookup[stateToIdx, Key[seedState], 1]` and falls back to index 1 if the seed is not found. This fallback cannot occur for well-formed inputs (the seed state is always in `allStates`) but provides a safe default.

Interaction with maxDepth. The BFS leaf set may be incomplete if any transition requires more than `maxDepth` bits (currently 22). An algorithm with a long-tailed random path (e.g., a geometric distribution with small success probability) might have its low-probability transitions truncated. In the current code, reaching `maxDepth` causes a hard error (exit) rather than a silent omission, so the ergodicity check cannot silently produce false `PASSes` from incomplete leaves.

7 New and Undocumented Issues

7.1 NormalDistribution discretisation: undocumented truncation error

The `RandomVariate[NormalDistribution[ $\mu$ ,  $\sigma$ ]]` handler discretises onto $\{\text{Round}(\mu) - \text{nMax}, \dots, \text{Round}(\mu) + \text{nMax}\}$ where $\text{nMax} = \text{Floor}[\$dbcCurrentNGrid/2]$. For a 3×3 grid this gives $\text{nMax} = 1$ and only 3 values: $\{\mu - 1, \mu, \mu + 1\}$. The probability mass outside this range is discarded, and the weights are renormalised.

This introduces a systematic error:

σ	Mass covered	Mass discarded
0.5	99.7%	0.3%
1.0	86.6%	13.4%
2.0	54.7%	45.3%

The renormalisation ensures the BFS leaf weights still sum to 1, but the transition probabilities assigned to each outcome are *wrong*: the checker verifies DB for a truncated distribution, not for the actual `NormalDistribution`. A user applying the checker to an algorithm that genuinely uses `RandomVariate[NormalDistribution[ $\mu$ ,  $\sigma$ ]]` with $\sigma \geq 1$ on a 3×3 grid would receive a DB verdict that is **not valid for the intended distribution**.

The correct fix is either to use the exact CDF-based weights (which is what the code already does for the truncated range) *and* to include a tail-capture term that catches all values outside $[\mu - \text{nMax}, \mu + \text{nMax}]$, or to document explicitly that the checker is approximating the distribution and that $\sigma \ll \text{nMax}$ is required for the approximation to be acceptable. The current code prints no warning, making this a silent correctness risk.

7.2 Coupling atom arity constraint

The validation check

```
!AllTrue[symCouplings, MatchQ[#, _[_Integer, _Integer, _Integer]]&]
```

requires every coupling atom to have *exactly* three integer arguments. An algorithm using two-argument couplings (`coupling[a,b]`, for distance-independent interactions) would fail with the message “coupling atom arguments must all be integers,” which does not explain the three-argument requirement. This is an unnecessary constraint: the coupling symmetry rule `h[b_, a_, rest__Integer] /; b > a := h[a, b, rest]` requires `rest` to match at least one argument, but a two-argument coupling could be supported with a separate rule. The error message should at minimum clarify that three integer arguments are required, or the constraint should be relaxed.

7.3 Legacy dead code

The functions `$dbcApplyRot90` and `$dbcApplyReflect` are defined in `dbc_core.wl` but never called in the current pipeline (the D4 action is computed everywhere via `$dbcApplyGElem` and `$dbcD4Pos`). These are artefacts of an earlier version and constitute dead code. Their presence is harmless but confusing; they should be removed.

7.4 RandomInteger with inverted bounds

As noted in Section 2, `RandomInteger[{hi, lo}]` with $hi < lo$ produces $n \leq 0$. The code silently discards all paths via `outOfRange` rather than raising a `cantHandle` error. A user whose algorithm contains an inverted-bound call due to a bug would not be informed by the checker. A guard

```
If[n <= 0, Throw[$dbc$cantHandle["RandomInteger:␣invalid␣range"],
  $dbc$tag]]
```

would make the failure visible.

8 Correctness of Orbit Expansion

8.1 The orbit map construction

In `$dbcComputeOrbits`, the canonical representative `minImg` is chosen as the lexicographically smallest state in the orbit. The orbit map `orbitMap[s] = g` satisfies $g(\text{minImg}) = s$. The comment in the source correctly identifies the bug that was fixed: when the initially-visited state $s \neq \text{minImg}$, the orbit map must be computed *from minImg*, not from s . The fix (recomputing `imgPairsFromRep` from `minImg`) is in place and correct.

8.2 Orbit expansion correctness

The expansion $T(sKey \rightarrow g(\text{nextState})) = T(\text{minImg} \rightarrow \text{nextState})$ relies on the G-invariance of the algorithm, which is established independently by the τ -check and D4 EBE check before the expansion is used. All states in `allStates` are correctly mapped under D4 actions (verified: zero misses for all 504 states and all 8 D4 elements for `single_metropolis`).

8.3 Self-loops and T-matrix diagonal

Pairs where $i = j$ (self-loops, *i.e.* the algorithm proposes a rejected or null move and returns the same state) are excluded from the DB check by `pairsArr = Select[pairsArr, #[[1]] != #[[2]] &]`. The DB condition $T(i \rightarrow i) \cdot \pi(i) = T(i \rightarrow i) \cdot \pi(i)$ is trivially satisfied, so this exclusion is correct.

9 Summary of Open Issues

The highest-severity open issue is the `NormalDistribution` truncation, which produces **wrong transition probabilities without warning** for $\sigma \geq 1$ on small grids. Algorithms using Gaussian displacement proposals should not be verified with this checker without explicitly confirming that $\sigma \ll \text{Floor}[\text{nGrid}/2]$.

Issue	Severity	Status	Affects examples?
\$dbcFS treated as violation	Moderate	Open	No
Chamber reuse assumption	Moderate	Open (undocumented)	No
NormalDistribution truncation	Significant	Open (undocumented)	No
Coupling arity constraint	Minor	Open	No
Inverted RandomInteger bounds	Minor	Open	No
Dead code (ApplyRot90 etc.)	Cosmetic	Open	No
D4 default performance	Performance	Fixed	Yes (vmmc_2d: 332s→217s)
Probabilistic SZ fallback	Correctness	Fixed	No

Table 1: Issues after both previous revision rounds, plus performance findings. “Affects examples?” refers to the nine provided example algorithms. The bottom two rows describe changes made in the current revision.

10 Performance Analysis: The D4 Orbit-Reduction Trade-off

10.1 Background and motivation

The original rationale for including "D4" in `$symmetryGroup` was that, if D4 symmetry holds, orbit representatives can be reduced from 56 (translation only) to 8 (D4+translation). Fewer representatives were expected to reduce the number of state pairs in EBE Phase 3 by a factor of 7, making the DB check $7\times$ faster. This section shows that this reasoning was quantitatively incorrect for VMMC, and documents the resulting architectural change.

10.2 Timing breakdown with and without D4

Comprehensive profiling of `vmmc_2d.w1` ($n = 3$, three particle types) produced the following wall-clock breakdown.

Original behaviour (D4 included in `$symmetryGroup`).

Step	Time	% of total
τ -BFS (56 reps)	12.8s	4%
BFS (56 reps)	11.7s	4%
D4 EBE check (Phase 2 + Phase 3 $\times$ 2 generators)	203.3s	61%
BFS (8 D4-reduced reps)	1.9s	<1%
Ergodicity	1.9s	<1%
DB EBE check (Phase 3, 8-rep leaves)	101.1s	30%
Total	332s	

New behaviour (D4 removed; translation only).

Step	Time	% of total
τ -BFS (56 reps)	13.9s	6%
BFS (56 reps)	12.8s	6%
D4 EBE check	0s	—
Ergodicity	2.0s	1%
DB EBE check (Phase 3, 56-rep leaves)	181.7s	84%
Total	217s	

The runtime decreases from 332s to 217s, a **1.53 $\times$ speedup**, by removing two lines from the algorithm file. For `single_metropolis.wl` ($k = 18$, 132 chambers) the improvement is from ≈ 65 s to ≈ 45 s.

10.3 Why D4 orbit reduction barely helps Phase 3

The expected $7\times$ speedup from D4 orbit reduction did not materialise because the cost of EBE Phase 3 is dominated by the number of *communicating state pairs*, not the number of orbit representatives.

The pair array `pairsArr` is computed by expanding orbit-rep leaves through the full orbit map and deduplicating by symmetry. With 56 translation-only reps, the translation group of size 9 is used for deduplication; with 8 D4+translation reps, the group of size 72 is used. The resulting pair counts are:

- 56 translation reps $\rightarrow$ 5,688 distinct pairs (up to translation)
- 8 D4+translation reps $\rightarrow$ 5,688 distinct pairs (up to D4+translation)

The pair count is *identical*. Both representations cover all 504 states and the same set of nonzero transitions; what changes is only the group used to deduplicate. Since the state space is fully ergodic and the transition graph is dense, every pair reachable from any single rep is also reachable from any other, so deduplication by a larger group yields the same universal set of pairs.

The only Phase 3 cost that differs between 56- and 8-rep runs is Step A, the J^* substitution into leaf weights: 4,368 leaves (56-rep) versus 648 leaves (8-rep). Empirically this costs ≈ 164 ms/region versus ≈ 17.5 ms/region — a $9.4\times$ increase for a $6.7\times$ increase in leaf count. Over 512 regions this is 74.7s extra, not the $7\times$ saving on the pair loop that was expected.

10.4 Why the D4 check appeared cheap (a subtle asymmetry)

The D4 EBE check also runs Phase 3 with 56-rep leaves, but costs only ≈ 203 s despite checking 2 generators instead of 1. This is because its per-pair work is structurally much lighter than the DB check. In the D4 check, the test expression is $T(i \rightarrow j) - T(Ri \rightarrow Rj)$. For a D4-symmetric algorithm these two T-values have identical symbolic structure after J^* substitution, so the `Expand` step reduces the expression to 0 immediately and `$dbcIsExpZero` is never called. In the DB check, the expression $T(i \rightarrow j) \cdot E^{-\beta E_i} - T(j \rightarrow i) \cdot E^{-\beta E_j}$ requires a full rational-coefficient grouping by Boltzmann exponent on every non-trivial pair. The D4 Phase 3 benefits from early termination in a way the DB Phase 3 cannot.

10.5 Implication: D4 is not a performance optimisation for dense transition graphs

The D4 orbit reduction was designed around a cost model that assumes Phase 3 scales with the number of orbit representatives. That assumption holds when the transition graph is sparse (each rep connects to a small, non-overlapping neighbourhood). For ergodic algorithms on a 3×3 grid the transition graph is dense and the assumption fails.

D4 verification remains available as an opt-in correctness check — useful when one specifically wants to verify rotational symmetry of the algorithm — but should **not** be included in `$symmetryGroup` for performance reasons.

10.6 The probabilistic fallback: permanently disabled

The `ebeMaxK` parameter previously defaulted to 50, which would trigger a probabilistic Schwartz-Zippel fallback for any algorithm with more than 50 Piecewise conditions. This default has been raised to 10,000 (effectively unlimited) to ensure the checker always runs the exact EBE algorithm. To use the probabilistic path explicitly, pass `-mode SZPure` or lower `-ebeMaxK` on the command line.

11 Python and Open-Source Portability Analysis

11.1 Motivation and scope

SZ-DBC is implemented in Wolfram Mathematica, which requires a proprietary licence. This severely limits adoption. The previous `report.tex` discussed language portability at a high level; the codebase has since changed substantially (FullSimplify is no longer the primary DB check, having been replaced by EBE). This section provides a detailed, up-to-date analysis of what can and cannot be ported to Python, Julia, or C++.

11.2 The five core mechanisms and their portability

11.2.1 Mechanism 1: Dynamic function interception (Block)

`Block[{RandomReal = f, RandomInteger = g, ...}, alg[state]]` rebinds every call to `RandomReal` and `RandomInteger` in the dynamic scope of `alg[state]`, regardless of call depth. The algorithm under test requires no modification.

Python. Python provides no equivalent at the standard language level. `unittest.mock.patch` can intercept named function calls, but only for module-level attributes, not for arbitrary call trees. Crucially, if the algorithm calls `import random; random.random()`, patching `random.random` in the test harness does not intercept calls made through a different import path. Algorithms using NumPy (`np.random.rand()`) or multiple RNG sources cannot be intercepted uniformly. Any call that escapes the interception returns a concrete float, and the BFS silently produces an incorrect leaf weight with no warning.

The workaround — requiring the algorithm to accept an explicit RNG object — is the only viable Python strategy, but it is *invasive*: the algorithm must be rewritten to pass `rng=symbolic_rng` through all function calls. This changes the algorithm’s interface, introduces a risk of translation errors, and breaks the key property that “the algorithm requires no modification.”

Julia. `Cassette.jl` provides a true call-tree overdubbing mechanism. A `Cassette.@overdub ctx rand()` call can redirect every invocation of `rand()` (and `rand(T)`, `rand(rng, ...)`) at the LLVM-IR level, regardless of call depth or module. This is semantically equivalent to Mathematica’s `Block`. It is the single most important feature for a rigorous port. However, `Cassette.jl` is a complex, low-level package with a steep learning curve and occasional compilation instability on newer Julia versions.

C++. No standard mechanism exists. One can compile the algorithm with a special RNG header that is swapped at link time, but this requires build-system cooperation and prevents use of third-party RNG implementations.

11.2.2 Mechanism 2: Operator overloading for comparison tokens

When `RandomReal[]` is intercepted, the returned symbolic token `$dbc$irand[j, acceptTestI]` must participate in comparison expressions like `token < Exp[- $\beta$  dE]`. In Mathematica, this is handled via *UpValues*: pattern-matching rules attached to the token head that trigger when the token appears in a `Less` expression.

Python. This is portable. A Python class `IntervalToken` can define `__lt__(self, other)` to call the BFS’s `accept_test` method, returning `True` or `False` after reading a bit and updating the weight. The algorithm’s conditional `if token < threshold:` triggers `__lt__` transparently. This works for direct comparisons; arithmetic (`token + c < threshold`) requires additional `__add__` overloading.

Julia. Also portable via method overloading: `Base.:<(token::IntervalToken, p)` is syntactically analogous to the Mathematica `UpValue`.

C++. Operator overloading is native to C++. The `operator<` method on a custom `IntervalToken` type works transparently.

11.2.3 Mechanism 3: Symbolic algebra for Piecewise expressions

After the BFS, leaf weights are symbolic expressions involving `Piecewise` conditions on coupling constants and Boltzmann factors $E^{-\beta E_i}$. The EBE Phase 1 extracts these conditions, Phase 2 enumerates chambers, and Phase 3 substitutes rational representatives and evaluates the `$dbcIsExpZero` check.

\$dbcIsExpZero in Python. This specific check (*group by $E^{-\beta r}$ exponent, sum rational coefficients, verify zero*) requires only rational arithmetic and dictionary operations. After substituting rational J^* values, all E_i are rational and all c_i are rational. This is fully implementable in Python using `fractions.Fraction` and a dict keyed by the rational exponent. **This is the easiest component to port.**

Piecewise handling. During the BFS (before J^* substitution), leaf weights contain nested `Piecewise` expressions. Mathematica’s `PiecewiseExpand` automatically computes the Cartesian product of all `Piecewise` conditions in a product expression. SymPy’s `piecewise_fold` does not perform this cross-product expansion; it only flattens nested `Piecewise` into the same `Piecewise`, not products of two independent `Piecewise`. A Python port must implement cross-product expansion manually, which is a non-trivial algebraic operation.

Condition feasibility. In Phase 2, Mathematica’s `FindInstance[constraints, vars, Rationals, 1]` is a complete decision procedure for systems of linear rational inequalities, returning an exact rational point. This has no Python equivalent. SymPy’s `Poly.all_roots` and `solve_univariate_inequality` handle some cases but are not complete for multivariate systems. The most practical replacement:

1. Use `scipy.optimize.linprog` to find a floating-point interior point.
2. Convert to a rational approximation using `fractions.Fraction.limit_denominator(N)` for large N .
3. Verify the rational point satisfies all strict inequalities.

This is an engineering solution that works in practice but can fail in degenerate cases (e.g., near the boundary of a very narrow chamber). Mathematica’s `FindInstance` gives an *exact* rational point by Fourier-Motzkin elimination.

Symbolic expression maintenance. During the BFS, weights are products of acceptance probabilities, which involve Piecewise expressions in the coupling constants. Mathematica’s pattern-matching evaluator automatically simplifies these products; SymPy requires explicit calls to `simplify` or `cancel` after each multiplication. For complex algorithms (e.g., VMMC), the symbolic expressions can grow very large; Mathematica’s evaluator handles these via efficient internal normal forms that SymPy cannot match.

11.2.4 Mechanism 4: State enumeration and orbit computation

These are combinatorial algorithms with no symbolic algebra.

Python, Julia, C++. All fully portable. `itertools.permutations` (Python), comprehensions (Julia), or STL containers (C++) handle state enumeration. The orbit computation is graph-theoretic and translates directly.

11.2.5 Mechanism 5: EBE Phase 2 chamber enumeration

The chamber-adjacency BFS is a graph traversal algorithm. The only Mathematica dependency is `FindInstance` (discussed above). The BFS logic itself is trivially portable.

11.3 Component-by-component portability summary

11.4 What the removal of `FullSimplify` changes

The original code (as described in `report.tex`) used Mathematica’s `FullSimplify` as the primary DB check. This was the component most deeply tied to Mathematica’s proprietary algebra engine: no other CAS matches Mathematica’s `FullSimplify` in breadth or reliability for mixed exponential-Piecewise expressions.

The current code uses EBE as the primary check, with `FullSimplify` available only as an optional mode (`-mode FullSimplify`). The EBE check’s inner computation (`$dbcIsExpZero`) is pure rational arithmetic — no CAS required. **This is the most significant portability improvement.** The EBE strategy makes the “heavy CAS” component optional rather than mandatory.

However, the EBE approach introduces its own Mathematica dependency: `FindInstance` for chamber enumeration. Replacing this with an approximate LP-based solution degrades

Component	Python	Julia	C++	Notes
Block interception	No	Via Cassettes.jl	Template mock	Key limitation
Operator UpValues	Yes	Yes	Yes	Straightforward
\$dbcIsExpZero	Yes	Yes	Yes	Pure rational arith.
PiecewiseExpand	Partial	Partial	No	Manual cross-product
FindInstance	Approx.	Approx.	Via LP solvers	Not exact
State enumeration	Yes	Yes	Yes	Trivial
Orbit computation	Yes	Yes	Yes	Trivial
BFS engine (queue)	Yes	Yes	Yes	Trivial
Coupling symmetry rule	Yes	Yes	Yes	Trivial
seqBernoulli	Yes	Yes	Yes	Straightforward

Table 2: Portability of each SZ-DBC component. “Yes” = direct port possible. “Partial” = feasible with additional implementation effort. “Approx.” = feasible with reduced exactness guarantees. “No” = fundamental limitation.

the exactness guarantee. The SZ fallback (probabilistic check at random coupling points) is fully portable but sacrifices the completeness guarantee.

11.5 A realistic Python port: what would it look like?

A realistic Python port of SZ-DBC would have the following structure:

1. **Algorithm interface:** The algorithm must be written (or rewritten) to accept an explicit RNG object. A `SymbolicRNG` class provides `random_real()` (returning an `IntervalToken`), `random_choice(items)` (using `seqBernoulli` or rejection sampling), and `random_integer(lo, hi)` (rejection sampling).
2. **BFS engine:** A Python queue manages bit-string prefixes. The algorithm is called with each prefix; the `IntervalToken` class’s operator overloads read bits and accumulate weights.
3. **Symbolic leaf weights:** After BFS, leaf weights are Python `Piecewise` objects (a list of {condition, value} pairs with a default). The condition is a Python expression tree over the coupling atoms.
4. **EBE Phase 1:** Traverse all leaf weights and collect unique conditions (linear inequalities in coupling atoms) using symbolic expression equality. SymPy can do this, albeit slowly for large expression trees.
5. **EBE Phase 2:** Chamber BFS using `scipy.optimize.linprog` for interior-point finding, with rational conversion.
6. **EBE Phase 3:** Substitute rational J^* into leaf weights; evaluate `Piecewise` conditions; group by exponent; sum rational coefficients. This is pure Python with `fractions.Fraction`.

Limitations of this port:

- The algorithm must be rewritten to use the explicit RNG interface. Any algorithm that calls Python’s `random` or NumPy’s RNG directly would bypass the BFS. This is both a usability burden and a correctness risk (missed random calls produce silently wrong transition probabilities).
- EBE Phase 2 is approximate (LP-based rational approximation), not exact. Chamber boundaries in very narrow regions might be misclassified.
- Piecewise cross-product expansion must be implemented manually. SymPy’s `piecewise_fold` is insufficient.
- The Python port would be significantly slower than the Mathematica version due to Python’s interpreted overhead and SymPy’s slower CAS.

11.6 Julia: the recommended open-source alternative

Julia with `Cassette.jl` + `Symbolics.jl` is the most viable open-source implementation, for the following reasons:

1. **Cassette.jl overdubbing** provides a true Block-like interception mechanism for `rand()` at any call depth. The algorithm requires zero modification. This is the single most important feature.
2. **Multiple dispatch** in Julia is semantically equivalent to Mathematica’s `UpValue` system for operator overloading. Defining `Base.:<(token::IntervalToken, p)` is a direct translation.
3. **Symbolics.jl** provides symbolic algebra with significantly faster compile-time expression evaluation than SymPy. `substitute` and `expand` handle the rational-coefficient Boltzmann algebra.
4. **\$dbcIsExpZero** translates directly using Julia’s `Dict` and `Rational{Int}` types.
5. **FindInstance** remains a gap. Julia’s `JuMP.jl` can find LP interior points; converting to rationals requires care. An exact alternative would be to implement Fourier-Motzkin elimination in Julia.

Remaining challenges in Julia:

- `Cassette.jl` has had API instability across Julia versions; the overdubbing context for `rand()` must be carefully constructed to handle all forms (`rand()`, `rand(Float64)`, `rand(rng, ...)`).
- `Symbolics.jl` has limited Piecewise support; cross-product expansion must be implemented from scratch.
- The `seqBernoulli` decomposition and interval-tracking logic must be reimplemented; no direct equivalent exists in any Julia library.

11.7 C++: not recommended

A C++ port would require a complete implementation of symbolic expression trees, Piecewise condition tracking, and rational arithmetic. While feasible (using e.g. the GiNaC computer algebra library), it would involve tens of thousands of lines of code with no natural analogue of Mathematica’s pattern-matching evaluator. The effort is not justified given the availability of Julia as a better alternative.

11.8 The fundamental limitation: algorithm transparency

The deepest limitation of any non-Mathematica port is that **the checker cannot guarantee that all random calls are intercepted**. In Mathematica, the Block mechanism intercepts every evaluation of the bound symbol, regardless of nesting. In Python, any call that bypasses the explicit RNG interface (e.g., a `random.random()` call inside a helper imported from a third-party library) returns a concrete float and produces a silently incorrect transition weight. In Julia (with Casset.jl), the overdubbing mechanism can intercept calls at the compiled bytecode level, providing a much stronger guarantee — but only if all random calls go through `rand()` or `Random.rand()`, not through e.g. a hand-written LCG in a C extension.

This issue is **not resolvable** by any finite amount of engineering: it is a consequence of Python’s (and C++’s) lack of dynamic rebinding at the language semantics level. In practice, it means that a Python port is inherently less trustworthy than the Mathematica version for algorithms that use multiple RNG sources or third-party libraries with their own random-number calls.

Mitigation. The Python port should include a mechanism to detect “escaped” random calls. One approach: wrap the algorithm in a context manager that monkey-patches `random.random`, `random.randint`, and `numpy.random.rand` (and their common aliases) at the module level, and fails loudly if any of these is called without going through the interceptor. This is fragile but provides a best-effort guarantee.

12 Recommendations

12.1 Immediate fixes

1. **Add NormalDistribution warning.** Before using the truncated discretisation, print: “WARNING: NormalDistribution discretised to $\pm n\text{Max}$ values around μ ; $X\%$ of probability mass discarded. Results are exact only for $\sigma \ll n\text{Max}$.”
2. **Fix \$dbcFS false-positive.** In the EBE Phase 3 check:

```
Which[res === True, Null,
      res === $dbcFS, Return[$dbc$cantHandle["$dbcIsExpZero_
      failed..."]],
      True, violations[pair] = ...]
```

3. **Add chamber-reuse subset assertion.** When reusing precomputed chambers, verify that the new BFS conditions form a subset:

```
If[!SubsetQ[allConds, precomputedConds],
```

```
Print["INTERNAL_ERROR: new conditions not covered by chambers"]; Exit[1]]
```

4. **Guard inverted RandomInteger bounds.** Add:

```
If[n <= 0, Throw[$dbc$cantHandle["RandomInteger: hi < lo"], $dbc$tag]]
```

5. **Improve coupling arity message.** Change the error message to explicitly state that three integer arguments are required.

12.2 Documentation

1. Document the open-chamber omission (measure-zero boundaries not checked).
2. Document that the τ -check verifies transition weights but not next-state correctness.
3. State explicitly that the NormalDistribution handler is an approximation.
4. Document the chamber-reuse assumption (D4 invariance implies condition subset, not verified at runtime).

12.3 For a Python or Julia port

1. **Julia (recommended):** Use `Cassette.jl` for RNG interception, `Symbolics.jl` for expression algebra, and implement `$dbcIsExpZero` and `seqBernoulli` from scratch. Accept LP-based (approximate) chamber enumeration as an engineering trade-off.
2. **Python:** Require the algorithm to accept an explicit `symbolic_rng` object; implement monkey-patching of common RNG module-level functions as a best-effort interception guard; use `fractions.Fraction` for exact coefficient arithmetic; and use `scipy.optimize.linprog` for chamber enumeration with rational conversion. Document that the interception guarantee is weaker than the Mathematica version.
3. In either language, the `$dbcIsExpZero` check (rational coefficient grouping by Boltzmann exponent) and the BFS queue management are the most straightforward components to port and should be implemented first as a proof-of-concept.

13 Conclusions

SZ-DBC in its current form is a sound and largely correct tool for verifying detailed balance in lattice MCMC algorithms. The critical bugs from the first audit have all been fixed. The EBE-based exact check is rigorously correct, the checker now always runs exactly (probabilistic SZ fallback requires explicit opt-in), and the D4 check has been moved to opt-in status following the performance analysis in Section 10. The result is a $1.53\times$ speedup for `vmmc_2d.w1` ($332s \rightarrow 217s$) with no loss of DB correctness.

The remaining open issues are:

- A moderate correctness concern around EBE chamber reuse (unverified assumption, mathematically justified but not checked at runtime).

- The `$dbcFS` sentinel being silently treated as a DB violation (harmless for all standard algorithms tested).
- A significant accuracy problem with the `NormalDistribution` discretisation (wrong transition probabilities for $\sigma \geq 1$, no warning printed).

For portability, the removal of `FullSimplify` from the critical path substantially improves the prospects for an open-source reimplementation. The EBE inner loop requires only rational arithmetic, which is available in Python, Julia, and C++. The fundamental bottleneck for Python remains the lack of a Block-equivalent for transparent RNG interception; Julia with `Cassette.jl` is the recommended path for an open-source port that preserves the checker’s non-invasive design.